

Chapter 6 Summary

Summary in Arabic

تلخيص الفصل السادس: SQL الأساسي (Basic SQL)

◆ مقدمة إلى SQL

- (لغة الاستعلام البنيوية) **Structured Query Language** تعني رسميًا ("تُنتَق" سيكويل) **SQL**.
- تُعد من الأسباب الرئيسية للنجاح التجاري للقواعد البيانات العلائقية (Relational Databases).
- نشأت من لغة رياضية تُسمى **Tuple Relational Calculus** (الحساب العلائقي بالبُنى)، وكانت تُعرف أولاً باسم **SQUARE**.
- تطورت إلى **SEQUEL** (في بحث Boyce و Chamberlin)، ثم اختصرت إلى **SQL** لغرض حقوق الملكية.
- رغم أن **SQL** ليست لغة برمجة كاملة، إلا أنها تُستخدم لتعريف البيانات، استرجاعها، التعديل عليها، وفرض القيود.
- ✓ ملاحظة: **SQL** تُعتبر "تطبيق عملي" للنموذج العلائقي، وليست صارمة رياضياً مثل نظرية المجموعات.

◆ مفاهيم تعريف البيانات وأنواعها في SQL

1. المصطلحات الأساسية

- جدول (Table) ↔ علاقة (Relation)
- صف (Row) ↔ بُنية (Tuple)
- عمود (Column) ↔ صفة (Attribute)

2. أوامر تعريف البيانات (DDL)

- الأمر الأساسي هو: **CREATE**
- يُستخدم لتعريف:
 - الجداول
 - القيود (Constraints)
 - العروض (Views)
 - الفهارس (Indexes)
 - الصلاحيات (Security)
 - قواعد البيانات الموزعة، الوسائط المتعددة، إلخ.

3. أنواع البيانات (Data Types)

النوع	الوصف	مثال
أعداد صحيحة	INTEGER , INT , SMALLINT	Age INT
أعداد عشرية	FLOAT , DOUBLE PRECISION	Salary FLOAT
سلاسل نصية	- CHAR(n) – ثابت الطول - VARCHAR(n) – متغير الطول	Name VARCHAR(50)
سلاسل بت (Bit)	BIT(n) , BIT VARYING(n)	Flags BIT(4)
قيمة منطقية	BOOLEAN (TRUE, FALSE, NULL)	IsActive BOOLEAN
التاريخ	DATE بصيغة YYYY-MM-DD	BirthDate DATE
الزمن الكامل	TIMESTAMP (يشمل التاريخ والوقت والجزء العشري للثانية)	LogTime TIMESTAMP
الفصل الزمني	INTERVAL – لتمثيل فترات زمنية (مثل 5+ أيام)	Duration INTERVAL DAY TO SECOND

✓ ملاحظة: يمكن تحويل أنواع التاريخ إلى نصوص للمقارنة، والعكس.

4. المجالات (Domains) وأنواع المستخدم (UDTs)

- يُستخدم لإنشاء نوع مخصص يُستخدم في عدة أعمدة: **DOMAIN**.

```
CREATE DOMAIN SSN_TYPE AS CHAR(9);
```

ثم نستخدمه:

```
CREATE TABLE EMPLOYEE (  
    Ssn SSN_TYPE,  
    ...  
);
```

✓ مزايا المجالات: سهولة التعديل، وتحسين قابلية القراءة.

- CREATE TYPE**: لدعم البرمجة الكائنية (Object-Oriented) في SQL-3.

◆ المخططات (Schemas) والفهارس (Catalogs)

- Schema (مخطط)**:

• يُعرف باسم ويحتوي على:

- الجداول
- القيود
- العروض
- المجالات
- مثال:

```
CREATE SCHEMA COMPANY AUTHORIZATION 'Jsmith';
```

• Catalog (فهرس):

- مجموعة من المخططات (Schemas).
- يُستخدم لتنظيم قواعد البيانات الكبيرة.
- ✓ ملاحظة: كل جملة SQL تنتهي بفاصلة منقوطة ; .

◆ تعريف الجداول باستخدام CREATE TABLE

1. أنواع الجداول

- جداول أساسية (Base Tables):
 - تُخزن فعليًا على القرص.
- جداول افتراضية (Virtual Tables / Views):
 - لا تُخزن، بل تُنشأ بـ CREATE VIEW .

2. مثال على إنشاء جدول (EMPLOYEE)

```
CREATE TABLE EMPLOYEE (
    Fname VARCHAR(15) NOT NULL,
    Minit CHAR,
    Lname VARCHAR(15) NOT NULL,
    Ssn CHAR(9) PRIMARY KEY,
    Bdate DATE,
    Address VARCHAR(30),
    Sex CHAR,
    Salary DECIMAL(10,2),
    Super_ssn CHAR(9),
    Dno INT,
    FOREIGN KEY (Super_ssn) REFERENCES EMPLOYEE(Ssn),
    FOREIGN KEY (Dno) REFERENCES DEPARTMENT(Dnumber)
);
```

- ✓ ملاحظة: يمكن تحديد الجدول ضمن مخطط: CREATE TABLE COMPANY.EMPLOYEE .

◆ القيود (Constraints) في SQL

1. أنواع القيود الأساسية

النوع	الوصف
قيود المفتاح (Key Constraint)	لا يمكن تكرار قيمة المفتاح الأساسي
سلامة الكيان (Entity Integrity)	المفتاح الأساسي لا يمكن أن يكون NULL
السلامة المرجعية (Referential Integrity)	القيمة في المفتاح الخارجي يجب أن توجد في الجدول المرجعي أو تكون NULL

2. تحديد القيود على الأعمدة

- العمود لا يقبل القيم الفارغة : NOT NULL
- قيمة افتراضية عند عدم التحديد : DEFAULT

```
Dno INT DEFAULT 1
```

- لفرض شرط على العمود : CHECK

```
Dnumber INT CHECK (Dnumber > 0 AND Dnumber < 21)
```

3. تحديد المفاتيح

- يحدد المفتاح الأساسي : PRIMARY KEY
- (Candidate Key) يحدد مفتاحاً بديلاً : UNIQUE

```
Dname VARCHAR(15) UNIQUE
```

- يربط بجدول آخر : FOREIGN KEY

```
FOREIGN KEY (Dno) REFERENCES DEPARTMENT(Dnumber)
```

4. إجراءات السلامة المرجعية (Referential Triggered Actions)

تُستخدم عند حذف أو تحديث سجل مرجعي:

- حذف السجلات التابعة تلقائيًا : ON DELETE CASCADE
- تعيين القيمة الخارجية إلى NULL : ON DELETE SET NULL
- تعيين قيمة افتراضية : ON DELETE SET DEFAULT

- `ON DELETE NO ACTION` (الافتراضي): رفض العملية إذا كانت هناك سجلات تابعة.

✓ **ملاحظة:** `CASCADE` مناسب للعلاقات القوية (مثل موظف ← إدارة)، بينما `SET NULL` مناسب للعلاقات الاختيارية.

5. تسمية القيود (Naming Constraints)

يمكن تسمية القيود لتسهيل التعديل لاحقًا:

```
CONSTRAINT FK_Employee_Dept FOREIGN KEY (Dno) REFERENCES DEPARTMENT(Dnumber)
```

6. قيود على مستوى السطر (Tuple-level CHECK)

- تُستخدم لفرض شروط على السطر ككل.
- `CREATE TABLE` توضع في نهاية.

```
CHECK (Dept_create_date <= Mgr_start_date)
```

◆ استعلامات الاسترجاع الأساسية (SELECT Queries)

1. هيكل الاستعلام: `SELECT-FROM-WHERE`

```
SELECT [DISTINCT] قائمة_الأعمدة
FROM قائمة_الجداول
WHERE شرط_التحديد;
```

2. أمثلة عملية

- الاستعلام 1: استرجاع أسماء الموظفين في القسم 5.

```
SELECT Fname, Lname
FROM EMPLOYEE
WHERE Dno = 5;
```

- الاستعلام 8: استرجاع اسم الموظف ورئيسه المباشر.

```
SELECT E.Fname, E.Lname, S.Fname, S.Lname
FROM EMPLOYEE AS E, EMPLOYEE AS S
WHERE E.Super_ssn = S.Ssn;
```

✓ ملاحظة: استخدمنا AS E و AS S لإنشاء "متغيرات صفية" (Tuple Variables) للإشارة إلى نفس الجدول مرتين.

3. أسماء الأعمدة المبهمة

- إذا كان عمود له نفس الاسم في جدولين، يجب تأهيله باسم الجدول:

```
SELECT EMPLOYEE.Name, DEPARTMENT.Name
FROM EMPLOYEE, DEPARTMENT
WHERE EMPLOYEE.Dno = DEPARTMENT.Dnumber;
```

4. استخدام النجمة (*)

- `SELECT * FROM EMPLOYEE;` → يعيد جميع الأعمدة
- `SELECT EMPLOYEE.* FROM EMPLOYEE, DEPARTMENT;` → يحدد الجدول تحديدًا دقيقًا

5. الجداول كمجموعات (Sets)

- أي تسمح بالتكرار، تُعامل الجداول كـ **Multisets** (SQL).
- لحذف التكرارات: استخدم `DISTINCT`

```
SELECT DISTINCT Salary FROM EMPLOYEE;
```

6. العمليات المجموعة (Set Operations)

الوصف	العملية
اتحاد (بدون تكرار)	UNION
اتحاد (مع تكرار)	UNION ALL
تقاطع	INTERSECT
فرق (في بعض الأنظمة: MINUS)	EXCEPT

✓ شرط: يجب أن تكون الجداول "متوافقة من الناحية النوعية" (Type Compatible).

7. مطابقة النمط (Pattern Matching)

- مع `LIKE`:
 - % → أي عدد من الأحرف (بما فيهم الصفر).
 - _ → حرف واحد فقط.

```
WHERE Address LIKE '%Houston,TX%'
```

```
WHERE Ssn LIKE '__1__8901' -- الحرف 3 هو 1، والحرف 6-8 هو 8901
```

8. العمليات الحسابية

- يمكن استخدام + , - , * , / في SELECT .

```
SELECT Fname, Lname, Salary * 1.1 AS NewSalary
FROM EMPLOYEE
WHERE Dno IN (SELECT Dnumber FROM DEPARTMENT WHERE Dname='Research');
```

9. ترتيب النتائج (ORDER BY)

- يُستخدم لفرز النتائج ORDER BY .
- ASC (تصاعدي) – الافتراضي .
- DESC (تنازلي) .

```
ORDER BY Dname DESC, Lname ASC, Fname ASC
```

◆ أوامر التعديل: INSERT, DELETE, UPDATE

1. INSERT (إدراج)

- لإضافة صفوف جديدة.

```
INSERT INTO EMPLOYEE VALUES ('Ahmed', 'M', 'Ali', '123456789', ...);
```

- أو بتحديد الأعمدة:

```
INSERT INTO EMPLOYEE (Fname, Lname, Ssn) VALUES ('Ali', 'Hassan',
'987654321');
```

- إدخال جماعي (Bulk Load):

```
INSERT INTO D5EMPS
SELECT * FROM EMPLOYEE WHERE Dno = 5;
```

✅ ملاحظة: يجب احترام القيود (مثل NOT NULL، PRIMARY KEY).

2. DELETE (حذف)

- يحذف صفوفًا من جدول واحد.

```
DELETE FROM EMPLOYEE WHERE Dno = 5;
```

- بدون WHERE : يحذف كل السجلات (يبقى الجدول فارغًا).

```
DELETE FROM EMPLOYEE; -- جدول فارغ
```

⚠ تحذير: لا يمكن التراجع بسهولة. يُفضل استخدام WHERE دائمًا.

3. UPDATE (تحديث)

- يُعدّل قيم الأعمدة.

```
UPDATE EMPLOYEE
SET Salary = Salary * 1.1
WHERE Dno IN (SELECT Dnumber FROM DEPARTMENT WHERE Dname = 'Research');
```

- يمكن تحديث أكثر من عمود:

```
UPDATE PROJECT
SET Plocation = 'Bellaire', Dnum = 5
WHERE Pnumber = 10;
```

✅ ملاحظة: القيمة على يمين = هي القيمة القديمة، وعلى اليسار هي الجديدة.

◆ ملاحظات عامة وتلخيص

المفهوم	ملاحظة
ليست صارمة رياضياً SQL	تسمح بالتكرار (Multiset) بخلاف النموذج العلاقي
القيود مهمة	تُحافظ على سلامة البيانات
WHERE يتم بالـ JOIN	في هذا الفصل، الربط بين الجداول يكون عبر WHERE (سيتم شرح JOIN لاحقًا)
الاستعلامات متداخلة (Nested Queries)	تُستخدم في WHERE و FROM

المفهوم	ملاحظة
الأسماء المختصرة (Aliases)	تُستخدم لتبسيط الاستعلامات المعقدة

الخلاصة النهائية

الفصل السادس يقدم الأساسيات الكاملة لـ SQL، من:

- تعريف الجداول وأنواع البيانات
- فرض القيود (مفاتيح، تحقق، افتراضيات)
- استرجاع البيانات عبر `SELECT`
- تعديل البيانات عبر `INSERT` , `DELETE` , `UPDATE`

وهو حجر الأساس لفهم باقي الفصول التي تتناول الاستعلامات المتقدمة، العروض، والبرمجة مع SQL.

Summary in English

Summary of Chapter 6: Basic SQL

(Elmasri and Navathe – Fundamentals of Database Systems)

◆ 1. Introduction to SQL

- **SQL (Structured Query Language)** is one of the main reasons for the commercial success of relational database management systems (RDBMS).
- It originated from **tuple relational calculus**, initially proposed as a language called **SQUARE**.
- The name **SQL** comes from "**SEQUEL**" (Structured English Query Language), introduced in a paper by **Chamberlin and Boyce** at IBM.
- Due to trademark issues, "SEQUEL" was shortened to **SQL**, which is now officially pronounced as "S-Q-L" or informally as "sequel".
- SQL is a **practical and informal implementation** of the relational model, with syntax designed for ease of use in real-world applications.

 **Note:** Although SQL is based on relational theory, it does not strictly adhere to mathematical set theory (e.g., it allows duplicate rows).

◆ 2. SQL Standards

- SQL has evolved through several standardized versions:
 - **SQL-86 (SQL-1)**: First formal standard.
 - **SQL-92 (SQL-2)**: Major revision with enhanced features.
 - **SQL:1999 (SQL-3)**: Introduced support for **object-oriented features**, recursive queries, and user-defined types (UDTs).
 - **SQL:2006**: Added **XML support**.
 - **SQL:2008**: Included enhancements for **data analytics**, temporal data, and further object-relational features.

✓ **Important:** No RDBMS fully implements all features of SQL-3. Vendors often add proprietary extensions.

◆ 3. Schema and Catalog Concepts

✓ Schema

- A named container that holds database objects such as:
 - Tables
 - Views
 - Constraints
 - Domains
 - Indexes
- Defined using `CREATE SCHEMA`.

```
CREATE SCHEMA COMPANY AUTHORIZATION 'Jsmith';
```

- Each SQL statement ends with a semicolon (;).

✓ Catalog

- A named collection of **schemas**.
- Used to organize large databases with multiple users or applications.
- A system can have multiple catalogs, each containing multiple schemas.

✓ **Note:** Schemas help manage naming conflicts and access control in multi-user environments.

◆ 4. The CREATE TABLE Command

✓ Base Tables vs. Virtual Tables (Views)

Type	Description
Base Table	Physically stored in the database. Created with CREATE TABLE .
View (Virtual Table)	Not physically stored. Defined via CREATE VIEW as a saved query.

✓ Syntax Example: Creating the EMPLOYEE Table

```
CREATE TABLE EMPLOYEE (  
    Fname VARCHAR(15) NOT NULL,  
    Minit CHAR,  
    Lname VARCHAR(15) NOT NULL,  
    Ssn CHAR(9) PRIMARY KEY,  
    Bdate DATE,  
    Address VARCHAR(30),  
    Sex CHAR,  
    Salary DECIMAL(10,2),  
    Super_ssn CHAR(9),  
    Dno INT,  
    FOREIGN KEY (Super_ssn) REFERENCES EMPLOYEE(Ssn),  
    FOREIGN KEY (Dno) REFERENCES DEPARTMENT(Dnumber)  
);
```

✓ You can qualify the table with a schema: CREATE TABLE COMPANY.EMPLOYEE (...).

◆ 5. Data Types and Domains

✓ Basic Data Types in SQL

Category	Types	Description
Numeric	INTEGER , INT , SMALLINT FLOAT , REAL , DOUBLE PRECISION	For whole numbers and floating-point values
Character Strings	CHAR(n) – fixed length VARCHAR(n) – variable length	Use VARCHAR for efficiency when length varies
Bit Strings	BIT(n) , BIT VARYING(n)	For binary data like flags or bitmaps
Boolean	BOOLEAN	Values: TRUE , FALSE , NULL

Category	Types	Description
Date & Time	DATE (YYYY-MM-DD) TIME TIMESTAMP (includes fractional seconds) INTERVAL	For time-based calculations and durations

✓ Dates and timestamps can be converted to strings for comparison using casting functions.

✓ Domains

- A **domain** defines a reusable data type with constraints.
- Improves consistency and readability.

```
CREATE DOMAIN SSN_TYPE AS CHAR(9);
```

Then use it:

```
Ssn SSN_TYPE PRIMARY KEY
```

✓ User-Defined Types (UDTs)

- Supported via `CREATE TYPE` (introduced in SQL:1999).
- Enables object-oriented modeling (see Chapter 12).

◆ 6. Specifying Constraints in SQL

Constraints ensure **data integrity** and enforce business rules.

✓ Three Fundamental Constraints

1. **Key Constraint:** Primary key values must be unique.
2. **Entity Integrity:** Primary key cannot be `NULL`.
3. **Referential Integrity:** Foreign key must reference an existing primary key or be `NULL`.

✓ Attribute-Level Constraints

Constraint	Purpose	Example
NOT NULL	Prevents null values	Fname VARCHAR(15) NOT NULL
DEFAULT <value>	Sets default value	Dno INT DEFAULT 1
CHECK(condition)	Enforces domain rules	CHECK (Salary > 0)

Example:

```
Dnumber INT NOT NULL CHECK (Dnumber > 0 AND Dnumber < 21)
```

✓ Key and Referential Integrity Constraints

Clause	Purpose
PRIMARY KEY (attr)	Defines primary key
UNIQUE (attr)	Defines alternate (candidate) keys
FOREIGN KEY (attr) REFERENCES Table(pk)	Enforces referential integrity

✓ Referential Triggered Actions (ON DELETE / ON UPDATE)

Used when referenced row is deleted/updated:

| Action | Behavior |

|-----|-----|

| CASCADE | Delete/update related tuples automatically |

| SET NULL | Set foreign key to NULL |

| SET DEFAULT | Set foreign key to default value |

| NO ACTION | Reject operation if dependent rows exist (default) |

✓ Example:

```
FOREIGN KEY (Dno) REFERENCES DEPARTMENT(Dnumber)
ON DELETE SET NULL ON UPDATE CASCADE
```

✓ **Best Practice:** Use CASCADE for strong ownership relationships (e.g., department → employees), SET NULL for optional relationships.

✓ Naming Constraints

Use CONSTRAINT <name> for easier future modification:

```
CONSTRAINT FK_Emp_Dept FOREIGN KEY (Dno) REFERENCES DEPARTMENT(Dnumber)
```

✓ Tuple-Level Constraints with CHECK

- Apply conditions across multiple attributes in one row.
- Placed at the end of CREATE TABLE .

```
CREATE TABLE DEPARTMENT (  
    Dname VARCHAR(15),  
    Dept_create_date DATE,  
    Mgr_start_date DATE,  
    CHECK (Dept_create_date <= Mgr_start_date)  
);
```

◆ 7. Basic Retrieval Queries: SELECT-FROM-WHERE

The core structure for querying data:

```
SELECT [DISTINCT] attribute_list  
FROM table_list  
WHERE condition;
```

✓ Key Components

- **Projection:** Attributes listed in SELECT (what to retrieve).
- **Selection:** Boolean condition in WHERE (which rows to retrieve).
- **Join Condition:** When multiple tables are used, relate them via common attributes.

✓ Examples of Queries

◆ Query 1: Retrieve names of employees in department 5

```
SELECT Fname, Lname  
FROM EMPLOYEE  
WHERE Dno = 5;
```

◆ Query 8: Retrieve employee and supervisor names

```
SELECT E.Fname, E.Lname, S.Fname, S.Lname
FROM EMPLOYEE AS E, EMPLOYEE AS S
WHERE E.Super_ssn = S.Ssn;
```

- ✓ Uses **tuple variables (aliases)** `E` and `S` to refer to the same table twice.

✓ Ambiguous Attribute Names

- If two tables have the same column name (e.g., `Name`), qualify it with the table name:

```
SELECT EMPLOYEE.Name, DEPARTMENT.Name
FROM EMPLOYEE, DEPARTMENT
WHERE EMPLOYEE.Dno = DEPARTMENT.Dnumber;
```

✓ Aliasing (AS)

- Shorten table names or rename output columns.

```
FROM EMPLOYEE AS E
SELECT E.Fname AS First_Name
```

- ✓ `AS` keyword is optional in most RDBMSs.

◆ 8. Advanced Query Features

✓ Using the Asterisk (*)

- Retrieves all attributes:

```
SELECT * FROM EMPLOYEE;
```

- Can be qualified:

```
SELECT EMPLOYEE.* FROM EMPLOYEE, DEPARTMENT WHERE ...;
```

✓ Missing WHERE Clause

- Results in a **Cartesian Product** (cross join) of all tables.

```
SELECT * FROM EMPLOYEE, DEPARTMENT; -- All combinations!
```

✓ Duplicate Handling

- SQL treats tables as **multisets (bags)**, so duplicates are allowed.
- To remove duplicates, use `DISTINCT` :

```
SELECT DISTINCT Salary FROM EMPLOYEE;
```

✓ Set Operations

Operation	Meaning
UNION	Combine results, remove duplicates
UNION ALL	Combine results, keep duplicates
INTERSECT	Common rows
EXCEPT (or MINUS in Oracle)	Rows in first not in second

- ✓ Tables must be **type-compatible** (same number of columns, compatible data types).

✓ Pattern Matching with LIKE

- `%` → matches any string of zero or more characters.
- `_` → matches exactly one character.

```
WHERE Address LIKE '%Houston,TX%'
WHERE Ssn LIKE '__1__8901' -- 3rd digit is 1, last 4 digits are 8901
```

✓ Range Checking with BETWEEN

```
WHERE Salary BETWEEN 30000 AND 40000
-- Equivalent to: Salary >= 30000 AND Salary <= 40000
```

✓ Arithmetic Expressions

- Use `+`, `-`, `*`, `/` in `SELECT` .

```
SELECT Fname, Lname, Salary * 1.1 AS Increased_sal
FROM EMPLOYEE, WORKS_ON, PROJECT
```

```
WHERE Pname = 'ProductX' AND ...;
```

✓ **Sorting Results with ORDER BY**

- Default: ascending (ASC).
- Use DESC for descending.

```
ORDER BY Dname DESC, Lname ASC, Fname ASC
```

- ✓ Always placed at the end of the query.

◆ **9. Data Modification Commands**

✓ **INSERT**

- Adds new rows to a table.

```
INSERT INTO EMPLOYEE VALUES ('John', 'B', 'Doe', '123456789', ...);
```

- Or specify columns:

```
INSERT INTO EMPLOYEE (Fname, Lname, Ssn) VALUES ('Alice', 'Smith', '987654321');
```

- **Bulk Insert** from a query:

```
INSERT INTO D5EMPS  
SELECT * FROM EMPLOYEE WHERE Dno = 5;
```

- ✓ Must respect constraints (NOT NULL , PRIMARY KEY , etc.).

✓ **DELETE**

- Removes rows from a single table.

```
DELETE FROM EMPLOYEE WHERE Dno = 5;
```

- Without `WHERE` : deletes **all rows** (table becomes empty):

```
DELETE FROM EMPLOYEE; -- Caution!
```

- ✓ Referential integrity is enforced. Use `CASCADE` if needed.

✓ UPDATE

- Modifies values in selected rows.

```
UPDATE EMPLOYEE
SET Salary = Salary * 1.1
WHERE Dno IN (SELECT Dnumber FROM DEPARTMENT WHERE Dname = 'Research');
```

- Multiple attributes:

```
UPDATE PROJECT
SET Plocation = 'Bellaire', Dnum = 5
WHERE Pnumber = 10;
```

- ✓ The right side of `=` uses the **old value**; the left side becomes the **new value**.

◆ 10. Summary and Key Points

Topic	Key Takeaway
SQL Basics	Practical language for defining, querying, and modifying relational databases.
Data Types	Includes numeric, string, date/time, boolean, and bit-string types.
Domains & UDTs	Improve schema clarity and reusability.
Constraints	Essential for data integrity: keys, <code>NOT NULL</code> , <code>CHECK</code> , defaults.
Referential Actions	<code>CASCADE</code> , <code>SET NULL</code> , <code>SET DEFAULT</code> help manage relationships.

Topic	Key Takeaway
SELECT Queries	Built on <code>SELECT-FROM-WHERE</code> ; supports joins, pattern matching, arithmetic.
Duplicates	SQL allows duplicates; use <code>DISTINCT</code> to eliminate them.
Set Operations	<code>UNION</code> , <code>INTERSECT</code> , <code>EXCEPT</code> require compatible schemas.
Data Modification	<code>INSERT</code> , <code>DELETE</code> , <code>UPDATE</code> must respect integrity constraints.

Final Notes

- This chapter lays the **foundation** for all SQL operations.
- Later chapters expand on:
 - Complex queries (nested, correlated)
 - Joins (`INNER JOIN` , `LEFT JOIN`)
 - Views
 - Triggers and active databases
 - Advanced integrity and security

 **Tip:** Always test `DELETE` and `UPDATE` with a `SELECT` first to verify the target rows.
